+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Plotly, Bokeh y NVD3.js: Guía básica

Introducción

Bienvenidos a esta primera guía de profundización de la semana 2, aquí introduciremos la creación de gráficos 3D e interactivos con Plotly, Bokeh y NVD3. A su vez, iremos abordando cómo definir la estructura de datos necesaria para que cada librería convierta datos en visualizaciones interactivas amigables a la vista. Dejaremos abierta la posibilidad para que experimenten cual librería se adapta más a su estilo de programación y esperamos despertar un poco más el interés en el amplio mundo de la visualización de datos.

Contenido

1. Visualización de datos en 3D:

Si bien existen una gran variedad de librerías para la visualización de datos, algunas de ellas no cuentan con visualización en 3D. En el temario de esta semana, para la creación de gráficos 3D, hicimos uso de una librería abordada en la primera semana (Matplotlib). Ya que está nos permite construir gráficos en 3D de una manera bastante amigable y sencilla. Por lo tanto, retomando el ejemplo de la agencia inmobiliaria, podemos crear el gráfico 3D en un par de líneas de código Python:

```
import numpy as np
import matplotlib.pyplot as plt
# Comando mágico en Jupyter notebook para pintar el gráfico en
# la misma celda
%matplotlib inline
# Se define en conjunto de datos a visualizar
house_dataset = np.array([[1, 60, 1000],[1, 63, 1050],
                          [1, 66, 1300],[1, 70, 1400],
                          [1, 65, 1100],[1, 61, 1050],
```

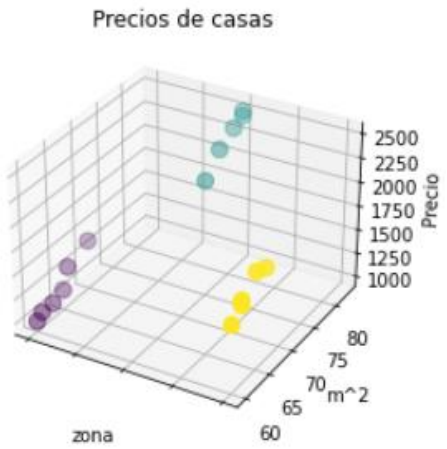
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
                          [2, 75, 2100],[2, 78, 2300],
                          [2, 83, 2500],[2, 83, 2450],
                          [2, 81, 2400],[3, 60, 1600],
                          [3, 62, 1700],[3, 65, 1900],
                          [3, 62, 1750],[3, 67, 1850]])
# Se crea una figura con un tamaño especificado en el figsize
fig = plt.figure(1, figsize=(4, 4))
# se crea el bosquejo del gráfico, especificando que tiene una
proyección en
# 3 dimensiones
ax = fig.add_subplot(111, projection="3d")
# Se utiliza el gráfico de dispersión, especificando las 3
dimensiones y se especifica que el primer valor del conjunto de datos
los clasifica (zona)
ax.scatter(house_dataset[:, 0],
    house_dataset[:, 1],
    house_dataset[:, 2],
    c=house_dataset[:, 0], s=80,)
# Se agrega fondo blanco al gráfico y etiquetado de ejes
ax.axes.set_facecolor(color="White")
ax.set_title("Precios de casas")
ax.set_xlabel("zona")
ax.xaxis.set_ticklabels([])
ax.set_ylabel("m^2")
ax.set_zlabel("Precio");
```

Es de resaltar que se importa el módulo numpy de computación científica, el cual permite crear la matriz con los datos de prueba, y el módulo de Matplotlib, que nos permite crear el gráfico 3D. Note que como en la semana anterior, se usaron marcadores y colores para obtener un mejor gráfico. Además, se usó un comando mágico para que el graficó generado por Matplotlib se viera dentro del Jupyter Notebook. Cómo resultado de la ejecución tendremos la siguiente gráfica:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 1: Gráfico 3D en Matplotlib



Fuente: Elaboración propia.

2. Gráficos interactivos con Plotly:

Una de las virtudes principales de Plotly, es que nos permite crear un gráfico interactivo con muy pocas líneas de código, aunque esto nos puede jugar en contra debido a que a veces no nos deja personalizar del todo ciertas características de los gráficos. Por ejemplo, bastan solo 4 líneas de código para graficar los datos referentes al precio del cierre de 6 diferentes acciones:

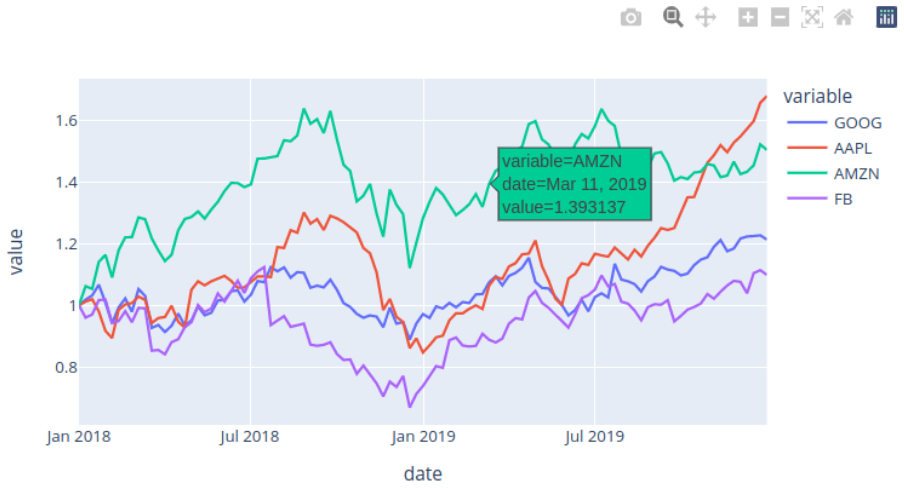
```
import plotly.express as px
# Cargamos el dataframe de la misma librería de plotly
df = px.data.stocks()
# Especificamos un gráfico de líneas conde el eje "x" tendrá las
fechas y el eje "y" los diferentes valores de las acciones.
fig = px.line(df, x='date', y=df.columns[1:5],width=700,
height=400)
# Se pinta la gráfica
fig.show()
```

Note que importamos la librería Plotly, de la cual hacemos uso para importar el conjunto de datos a graficar. A su vez, se puede evidenciar que Plotly no necesita que se le especifique directamente los valores a graficar en las variables *"x"* e *"y"*. Simplemente le pasamos como entrada el *DataFrame* llamado *"df"*, y especificamos que la columna *"date"* del *DataFrame* será el eje *"x"*, y la columna

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

"*y*" tomará los valores de las diferentes columnas. Como resultado tendremos el siguiente gráfico interactivo:

Figura 2: Gráfico Interactivo en Plotly.



Fuente: Elaboración propia.

Otra de las principales ventajas que tiene Plotly es que podemos crear un pequeño *dashboard* con pocas líneas de código, por ejemplo, si queremos correlacionar los precios de las acciones, podemos realizarlo mediante el siguiente código en Python:

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
app = Dash(__name__)
# Se agrega una capa la cual contiene la lista desplegable
# Se definen las opciones disponibles a seleccionar
app.layout = html.Div([
    html.H4('Análisis de matriz de dispersión'),
    dcc.Dropdown(
        id="dropdown",
        options=['GOOG', 'AAPL', 'AMZN', 'FB'],
        value=['GOOG', 'AMZN'],
        multi=True
    ),
    dcc.Graph(id="graph"),
])
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
@app.callback(
    Output("graph", "figure"),
    Input("dropdown", "value"))
def update_bar_chart(dims):
    # Cargamos el dataframe de la misma librería de plotly
    df = px.data.stocks()
    fig = px.scatter_matrix(
        df, dimensions=dims)
    return fig
app.run_server(debug=True)
```

En este ejemplo introducimos la librería Dash, la cual está desarrollada para conectarse con Plotly, y permitir agregar a nuestros gráficos más componentes interactivos como lo son en este caso las listas desplegables (*Fig. 2*). Mediante Dash creamos una aplicación, la cual contiene un título, una lista desplegable que permite seleccionar 4 acciones diferentes. Luego añadimos un *Graph*, que nos permitirá crear un espacio de gráfico interactivo.

Figura 3: Dashboard de correlación en Plotly

Fuente: Elaboración propia.

Note que dentro de nuestro código definimos una función *callback*, estás funciones son esenciales al momento de crear un gráfico interactivo. Lo que

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

hacen es dada una entrada de datos, retornar una nueva gráfica con el ajuste a los datos. Esta es una de las ventajas de usar conjuntos de datos como los *DataFrame*, ya que cuentan con información extra como nombre de columnas e índices, lo cual permite fácilmente decidir que se quiere graficar. Por ejemplo, si el usuario objetivo quiere ver la relación de las acciones de GOOG y FB, basta con seleccionar esas dos acciones de la lista desplegable, que la función callback se encargará de actualizar la gráfica.

3. Gráficos interactivos con Bokeh:

Bokeh es una de las librerías más utilizadas al momento de generar gráficos interactivos en Python, su mayor flexibilidad en la creación de gráficos es pieza clave, y a la vez es uno de sus retos, ya que a diferencia de Plotly, se necesita configurar un par de cosas extra para tener un gráfico interactivo básico. Por ejemplo, a diferencia de Plotly, si queremos visualizar los valores exactos de las acciones en el tiempo al pasar el cursor sobre ellas, debemos especificar a nivel del código dicha funcionalidad. Por ejemplo, en el siguiente bloque de código creamos un gráfico con dos precios de acciones:

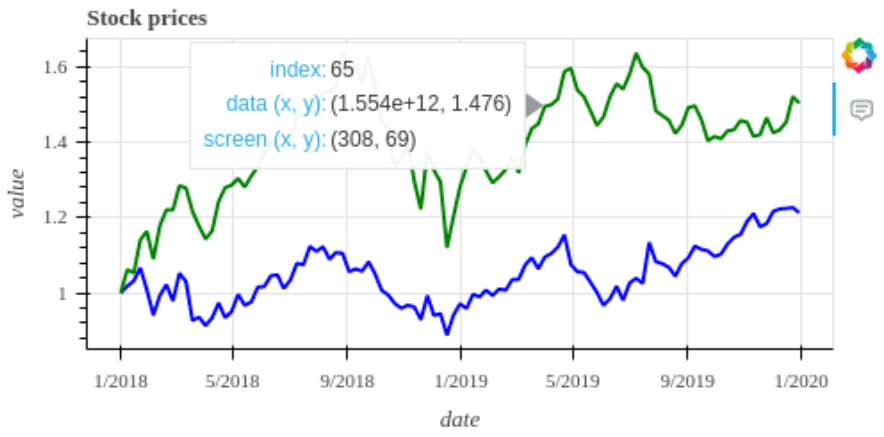
```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource
from bokeh.models import HoverTool
from bokeh.models import DatetimeTickFormatter
# Cargamos el conjunto de datos desde plotly
df = px.data.stocks()
# Es necesario formatear la fechas para que bokeh las interprete
como fechas
df["date"] = pd.to_datetime(df["date"])
# Se transforma el DataFrame a ColumnDataSource para aumentar la
eficiencia de renderización
source = ColumnDataSource(df)
# Se crea el gráfico y se configura un título y se etiquetan los
ejes.
p = figure(title="Stock prices", x_axis_type="datetime",
x_axis_label="date", y_axis_label="value",
          tools=[HoverTool()], max_width=500, height=250,)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# Se añaden los gráficos de líneas
p.line(x="date", y="GOOG", color="blue", line_width=2,
source=source)
p.line(x="date", y="AMZN", color="green", line_width=2,
source=source)
show(p)# Se muestra el gráfico
```

Note que a los gráficos no se les envía directamente los valores para el par de ejes *x* e *y*, en vez de eso, se adjunta un *source* el cual es el conjunto de datos a graficar (*ColumnDataSource)*, y haciendo uso del nombre de las columnas, se especifica qué columna se tome para ser representada por el eje *x* y que otra columna sea representada por el eje *y*. Una de las ventajas al usar Bokeh, es que a un mismo gráfico (llamado figura en Bokeh), le podemos añadir diferentes tipos de representaciones gráficas. En la Figura 4 podemos ver el gráfico generado por el bloque de código anteriormente mencionado. Note que la etiqueta que sale al pasar el cursor sobre la gráfica no es muy entendible a primera vista, en comparación a la etiqueta que genera por defecto Plotly.

Figura 4: visualización básica con Bokeh



Fuente: Elaboración propia.

Se puede observar que por defecto, el gráfico no permite identificar a qué acción pertenece cada gráfico de línea. Por consiguiente, vamos a explorar un poco más las opciones de configuración que nos brinda Bokeh. Para ello, definiremos una etiqueta para poder identificar a qué acción pertenece cada gráfico. A su vez,

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

modificaremos la etiqueta que pinta Bokeh por defecto para que nos resulte más amigable a la vista.

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource
from bokeh.models import HoverTool
from bokeh.models import DatetimeTickFormatter

# Cargamos el conjunto de datos desde plotly
df = px.data.stocks()
# Es necesario formatear la fechas
df["date"] = pd.to_datetime(df["date"])
# Se transforma el DataFrame a ColumnDataSource para aumentar la
eficiencia de renderización
source = ColumnDataSource(df)
# Definimos la apariencia de la leyenda interactiva a mostrar y
definimos la columna date
# como formato de fecha para que pueda ser interpretado
correctamente por bokeh
ht = HoverTool(
    tooltips=[
        ('date','@date{%F}'),
        ('GOOG','@GOOG'),
        ('AMZN','@AMZN'),
    ],
    formatters={
        '@date':'datetime',
    })
# Se crea el gráfico y se configura un título y se etiquetan los
ejes.
p = figure(title="Stock prices", x_axis_type="datetime",
x_axis_label="date", y_axis_label="value",
          tools=[ht, 'wheel_zoom, zoom_in, save, reset, pan'],
max_width=500, height=250)
# Se añaden los gráficos de líneas
p.line(x="date", y="GOOG", color="blue", line_width=2,
source=source, legend_label="GOOG")
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
p.line(x="date", y="AMZN", color="green", line_width=2,
source=source, legend_label="AMZN")
show(p) # Se muestra el gráfico
```

Lo que hicimos fue interactuar con el panel de *tools* de Bokeh, el cual nos permite especificar los datos a mostrar en el gráfico interactivo. Para este caso especificamos 3 datos importantes: La fecha, la cual tenemos que especificar mediante el *formatters* que es de tipo *datetime,* para que Bokeh la pueda interpretar de forma correcta y no coloque un número (*Fig. 4*). A su vez, especificamos los valores del precio de cierre de las dos acciones (GOOG y AMZN). Note que para identificar cada gráfico lineal, lo que hicimos fue agregar a cada uno la propiedad *legend_label*. Obteniendo el siguiente gráfico:

Figura 5: Interacción con las herramientas de visualización.

Fuente: Elaboración propia.

4. Gráficos interactivos con NVD3.js:

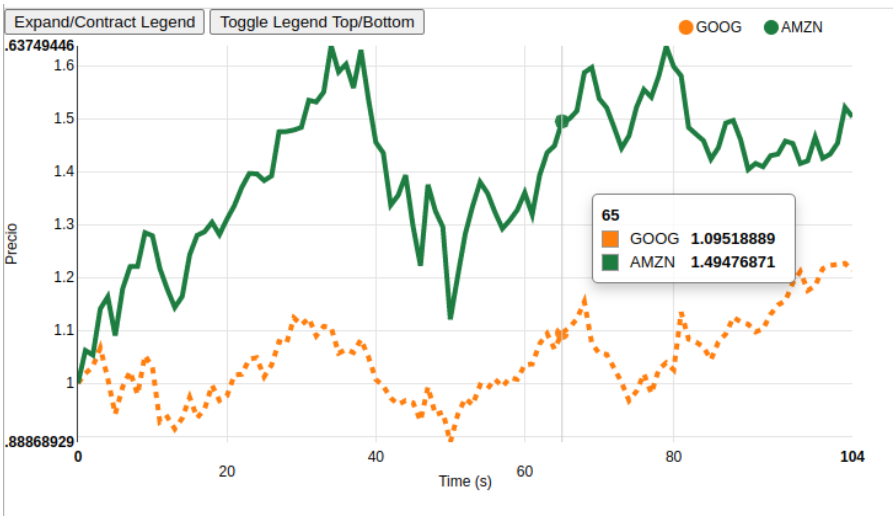
Como abordamos en el temario de la semana 2, NVD3 es una librería desarrollada en JavaScript la cual agrupa diferentes tipos de gráficos interactivos desarrollados en D3.js (tal vez la herramienta más poderosa para el diseño de gráficos interactivos). NVD3 nos brinda una abstracción, una capa más alta de programación de gráficos en JavaScript, para no tener que incurrir en el bajo nivel y la gran dificultad que puede implicar el crear un gráfico en D3.js. Para poder iniciar a graficar con NVD3.js es necesario contar con las dependencias descritas en la librería. Además de tener conocimientos básicos en HTML y CSS, para

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

poder visualizar en una página web los gráficos generados. Cabe resaltar que se tienen que tener conocimientos básicos de D3.js para poder generar ciertas interacciones. Pero como tal, NVD3.js nos brinda ya plantillas con diferentes tipos de gráficos interactivos, una gran ventaja en comparación de D3.js, pues en este último es necesario programar absolutamente todo.

Debido a la complejidad en la programación del código para generar un gráfico interactivo (*Fig. 6*), el código empleado será anexado al documento y nos enfocaremos en los detalles relevantes del código.

Figura 6: Visualización interactiva mediante NVD3.js.



Fuente: Elaboración propia.

Para el ejemplo en concreto, colocamos dentro del *script* HTML los datos a graficar. En un ambiente productivo, estos podrían ser retornados mediante un consumo a un API. Al fijarnos en el código veremos que el gráfico se añade a un *div* de HTML mediante la librería D3.js, a este se le pasan los datos en un formato de lista, donde cada posición representa un archivo JSON el cual contiene pares clave, valor que identifican la configuración de cada gráfico lineal a ser representado. Hasta ahora, vemos que utilizamos la librería D3.js para insertar el gráfico en la web. Entonces: ¿Cuál es el beneficio de NVD3.js? Este se puede identificar al momento de crear el gráfico a representar. NVD3.js cuenta con una lista de modelos ya programados en JavaScript en los cuales solo necesitamos configurar ciertos parámetros de entrada.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Note que para este ejemplo en concreto, seleccionamos de los modelos de NVD3.js un gráfico *lineChart*. Una vez definido el tipo de gráfica, lo visualizamos mediante D3.js, le pasamos por medio del método *datum* los datos a graficar, y mediante el método *call*, pasamos el gráfico *lineChart* definido anteriormente.

Conclusiones

Comprender los conceptos básicos para la visualización de gráficos interactivos en Plotly, Bokeh y NVD3.js: Tenga presente que tipo de gráfico quiere representar y cuál de las librerías abordadas le permitirá hacerlo en el menor tiempo posible. Se recomienda generar un gráfico base con Plotly, debido a que brinda la forma más sencilla de crear una visualización interactiva. Si quiere tener un poco más de libertad al diseñar, puede usar Bokeh. Y si quieres volverte un experto en visualización, ¡navega en el mundo de NVD3.js o D3.js!

Referencias

Novous Partners, (n.d.), NVD3 Re-usable charts for d3.js, recuperado el 14 de enero de 2024 de https://nvd3-community.github.io/nvd3/examples/documentation.html

Plotly, Inc. (n.d.), Dash Core components, Recuperado el 14 de enero de 2024 de https://dash.plotly.com/dash-core-components

Anaconda, Inc., and Bokeh Contributors, Bokeh First steps, (n.d.), Recuperado el 15 de enero de 2024 de https://docs.bokeh.org/en/latest/docs/first_steps.html#first-steps

Anexos

Anexo a este material se encuentra un archivo llamado d3_ stock.zip. Dentro de este encontrará una carpeta *build* la cual contiene los archivos necesarios para hacer uso de la librería NVD3.js. Para poder visualizar el gráfico interactivo, basta con abrir en el navegador de su preferencia el archivo *stock.html*. Para explorar el código, es necesario abrir el archivo en un editor de texto.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.